

1600. Throne Inheritance

Solved

Medium Topics Companies Hint

A kingdom consists of a king, his children, his grandchildren, and so on. Every once in a while, someone in the family dies or a child is born.

The kingdom has a well-defined order of inheritance that consists of the king as the first member. Let's define the recursive function `Successor(x, curOrder)`, which given a person `x` and the inheritance order so far, returns who should be the next person after `x` in the order of inheritance.

```
Successor(x, curOrder):
    if x has no children or all of x's children are in curOrder:
        if x is the king return null
        else return Successor(x's parent, curOrder)
    else return x's oldest child who's not in curOrder
```

For example, assume we have a kingdom that consists of the king, his children Alice and Bob (Alice is older than Bob), and finally Alice's son Jack.

1. In the beginning, `curOrder` will be `["king"]`.
2. Calling `Successor(king, curOrder)` will return Alice, so we append to `curOrder` to get `["king", "Alice"]`.
3. Calling `Successor(Alice, curOrder)` will return Jack, so we append to `curOrder` to get `["king", "Alice", "Jack"]`.
4. Calling `Successor(Jack, curOrder)` will return Bob, so we append to `curOrder` to get `["king", "Alice", "Jack", "Bob"]`.

322 19

23 Online

Code

Python Auto

```
1 from collections import defaultdict
2
3 class ThroneInheritance(object):
4
5     def __init__(self, kingName):
6         self.king = kingName
7         self.family_tree = defaultdict(list)
8         self.dead = set()
9
10    def birth(self, parentName, childName):
11        self.family_tree[parentName].append(childName)
```

Saved

Ln 27, Col 21

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Input

```
["ThroneInheritance","birth","birth","birth","birth","birth","birth","getInheritanceOrder","death","getInheritanceOrder"]
```

```
[["king"],["king","andy"],["king","bob"],["king","catherine"],["andy","matthew"],["bob","alex"],["bob","asha"],[null],["bob"],[null]]
```

Output

Generic methods are a very efficient way to handle multiple datatypes using a single method. This problem will test your knowledge on Java Generic methods.

Let's say you have an integer array and a string array. You have to write a **single** method `printArray` that can print all the elements of both arrays. The method should be able to accept both integer arrays or string arrays.

You are given code in the editor. Complete the code so that it prints the following lines:

```
1
2
3
Hello
World
```

Do not use method overloading because your answer will not be accepted.

```
32
33
34
```

Line: 34 Col: 1

📁 Upload Code as File

☐ Test against custom input

Run Code

Submit Code



You have earned 15.00 points!

You are now 10 points away from the 2nd star for your java badge.

60%

40/50

Congratulations

You solved this challenge. Would you like to challenge your friends? [🐦](#) [in](#)

Next Challenge

✅ Test case 0

Compiler Message

Success

Write the following code in your editor below:

1. A class named Arithmetic with a method named add that takes 2 integers as parameters and returns an integer denoting their sum.
2. A class named Adder that inherits from a superclass named Arithmetic.

Your classes should not be **public**.

Input Format

You are not responsible for reading any input from stdin; a locked code stub will test your submission by calling the add method on an Adder object and passing it 2 integer parameters.

Output Format

You are not responsible for printing anything to stdout. Your add method must return the sum of its parameters.

Sample Output

The main method in the Solution class above should print the following:

```
My superclass is: Arithmetic
42 13 20
```



You have earned 10.00 points!

You are now 25 points away from the 2nd star for your java badge.

0%

25/50

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

[Next Challenge](#)

Test case 0

Compiler Message

Success

Expected Output

[Download](#)

```
1 My superclass is: Arithmetic
2 42 13 20
```



Search



ENG
IN



09:52
02-09-2026

Using inheritance, one class can acquire the properties of others. Consider the following Animal class:

```
class Animal{
    void walk(){
        System.out.println("I am walking");
    }
}
```

This class has only one method, walk. Next, we want to create a Bird class that also has a fly method. We do this using extends keyword:

```
class Bird extends Animal {
    void fly() {
        System.out.println("I am flying");
    }
}
```

Finally, we can create a Bird object that can both fly and walk.

```
public class Solution{
    public static void main(String[] args){

        Bird bird = new Bird();
        bird.walk();
        bird.fly();
    }
}
```

```
25
26     Bird bird = new Bird();
27     bird.walk();
28     bird.fly();
29     bird.sing();
30
31 }
32 }
```

Line: 21 Col: 2



Upload Code as File



Test against custom input

Run Code

Submit Code



You have earned 5.00 points!

You are now 10 points away from the 1st star for your java badge.

60%

15/25

Congratulations

You solved this challenge. Would you like to challenge your friends?  

Next Challenge



Test case 0



Search



ENG

IN



09:52

02-09-2026

HackerLand University has the following grading policy:

- Every student receives a *grade* in the inclusive range from 0 to 100.
- Any *grade* less than 40 is a failing grade.

Sam is a professor at the university and likes to round each student's *grade* according to these rules:

- If the difference between the *grade* and the next multiple of 5 is less than 3, round *grade* up to the next multiple of 5.
- If the value of *grade* is less than 38, no rounding occurs as the result will still be a failing grade.

Examples

- *grade* = 84 round to 85 (85 - 84 is less than 3)
- *grade* = 29 do not round (result is less than 38)
- *grade* = 57 do not round (60 - 57 is 3 or higher)

Given the initial value of *grade* for each of Sam's *n* students, write code to automate the rounding process.

Function Description

Complete the function *gradingStudents* with the following parameter(s):

- *int grades[n]*: the grades before rounding

Returns

- *int[n]*: the grades after rounding

Input Format

The first line contains a single integer, *n*, the number of students.

```
20 if __name__ == '__main__':
21     fptr = open(os.environ['OUTPUT_PATH'], 'w')
22
23     grades_count = int(input().strip())
24
25     grades = []
26
```

Line: 36 Col: 17



Upload Code as File



Test against custom input

Run Code

Submit Code



You have earned 10.00 points!

You are now 30 points away from the 3rd star for your problem solving badge.

70%

170/200

Congratulations

You solved this challenge. Would you like to challenge your friends?



Next Challenge



Test case 0

Compiler Message



Search



ENG

IN



09:53

02-09-2026

1603. Design Parking System

Solved

Easy Topics Companies Hint

Design a parking system for a parking lot. The parking lot has three kinds of parking spaces: big, medium, and small, with a fixed number of slots for each size.

Implement the `ParkingSystem` class:

- `ParkingSystem(int big, int medium, int small)` Initializes object of the `ParkingSystem` class. The number of slots for each parking space are given as part of the constructor.
- `bool addCar(int carType)` Checks whether there is a parking space of `carType` for the car that wants to get into the parking lot. `carType` can be of three kinds: big, medium, or small, which are represented by 1, 2, and 3 respectively. **A car can only park in a parking space of its `carType`.** If there is no space available, return `false`, else park the car in that size space and return `true`.

Example 1:

Input

```
["ParkingSystem", "addCar", "addCar", "addCar", "addCar"]
[[1, 1, 0], [1], [2], [3], [1]]
```

Output

```
[null, true, true, false, false]
```

Explanation

```
ParkingSystem parkingSystem = new ParkingSystem(1, 1, 0);
parkingSystem.addCar(1); // return true because there is 1 available slot for a
```

2.1K 71

70 Online

Code

Python Auto

```
1 class ParkingSystem(object):
2
3     def __init__(self, big, medium, small):
4
5         self.spaces = [0, big, medium, small]
6
7     def addCar(self, carType):
8
9         if self.spaces[carType] > 0:
10             self.spaces[carType] -= 1
11             return True
```

Saved

Ln 4, Col 9

Testcase Test Result

You must run your code first

The Java instanceof operator is used to test if the object or instance is an instanceof the specified type.

In this problem we have given you three classes in the editor:

- Student class
- Rockstar class
- Hacker class

In the main method, we populated an ArrayList with several instances of these classes. count method calculates how many instances of each type is present in the ArrayList. The code prints three integers, the number of instance of Student class, the number of instance of Rockstar class, the number of instance of Hacker class.

But some lines of the code are missing, and you have to fix it by modifying only 3 lines! Don't add, delete or modify any extra line.

To restore the original code in the editor, click on the top left icon in the editor and create a new buffer.

Sample Input

```
5
Student
Student
Rockstar
Student
Hacker
```

Sample Output

```
26 Scanner sc = new Scanner(System.in);
27 int t = sc.nextInt();
28 for(int i=0; i<t; i++){
29     String s = sc.next();
30     if(s.equals("Student"))mylist.add(new Student());
31     if(s.equals("Rockstar"))mylist.add(new Rockstar());
32     if(s.equals("Hacker"))mylist.add(new Hacker());
33 }
34 System.out.println(count(mylist));
35 }
36 }
```

Line: 36 Col: 2

Upload Code as File

☐ Test against custom input

Run Code

Submit Code



You have earned 10.00 points!

You are now 15 points away from the 1st star for your java badge.

40%

10/25

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

Problem List

Submit

00:54:34

Premium

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 57 / 57 testcases passed

srikiran57 submitted at Sep 02, 2026 09:40

Analysis

Solution

₹583.25/mo

Save over 56%

Unlock the Full LeetCode Experience

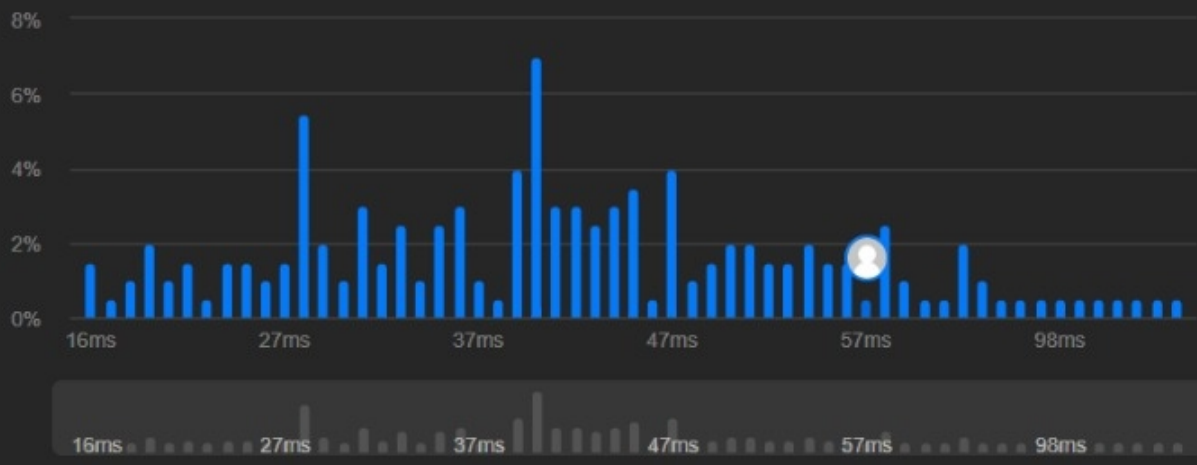
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

57 ms | Beats 18.41%

Memory

23.24 MB | Beats 47.76%



Code

Python

1 class UndergroundSystem(object):

Python

Auto

self.journeys[route] = [0, 0]

self.journeys[route][0] += travel_time

self.journeys[route][1] += 1

def getAverageTime(self, startStation, endStation):

route = (startStation, endStation)

total_time, trip_count = self.journeys[route]

return float(total_time) / trip_count

Testcase

Test Result

[[100, 10, "Leyton", 24], [100, "Waterloo", 10, "Waterloo", 30], [100, "Waterloo"]]

Output

[null, null, null, null, null, null, null, 14.00000, 11.00000, null, 11.00000, null, 12.00000]

Expected

[null, null, null, null, null, null, null, 14.00000, 11.00000, null, 11.00000, null, 12.00000]

Contribute a testcase

Search

09:53 02-09-2026

1472. Design Browser History

Solved

Medium
Topics
Companies
Hint

You have a **browser** of one tab where you start on the `homepage` and you can visit another `url`, get back in the history number of `steps` or move forward in the history number of `steps`.

Implement the `BrowserHistory` class:

- `BrowserHistory(string homepage)` Initializes the object with the `homepage` of the browser.
- `void visit(string url)` Visits `url` from the current page. It clears up all the forward history.
- `string back(int steps)` Move `steps` back in history. If you can only return `x` steps in the history and `steps > x`, you will return only `x` steps. Return the current `url` after moving back in history **at most** `steps`.
- `string forward(int steps)` Move `steps` forward in history. If you can only forward `x` steps in the history and `steps > x`, you will forward only `x` steps. Return the current `url` after forwarding in history **at most** `steps`.

Example:

Input:

```
["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]
[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],[7]]
```

4.2K
100
34 Online

Code

Python Auto

```
14
15     def back(self, steps):
16         self.curr = max(0, self.curr - steps)
17         return self.history[self.curr]
18
19     def forward(self, steps):
20         self.curr = min(self.bound, self.curr + steps)
21         return self.history[self.curr]
```

Saved

Ln 21, Col 39

Testcase Test Result

You must run your code first

Problem List

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 29 / 29 testcases passed

srikiran57 submitted at Sep 02, 2026 09:42

AnalysisSolution

₹583.25/mo

Save over 56%

Unlock the Full LeetCode Experience

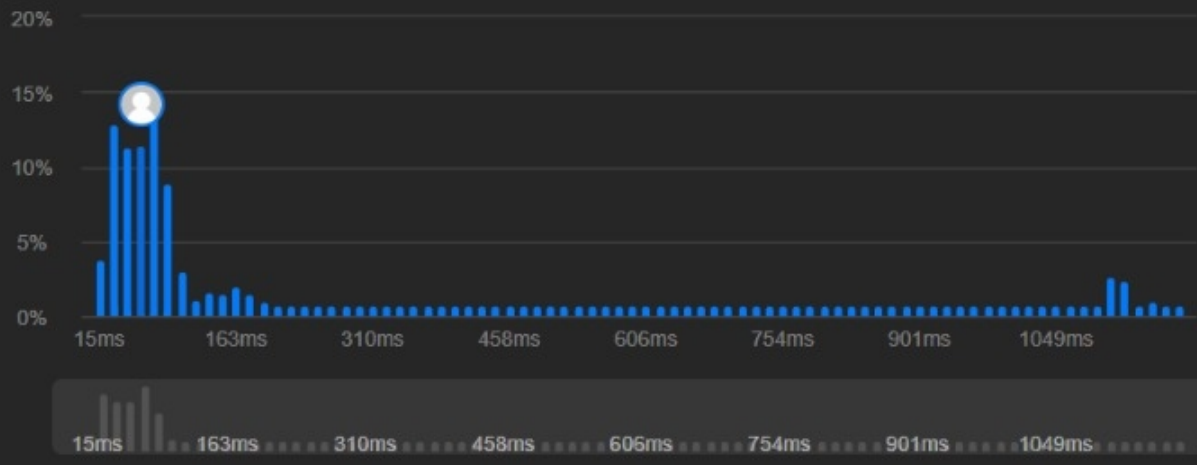
Company problems, Ask Leet, and expert editorials — all in one plan.

Runtime

67 ms | Beats 62.63%

Memory

37.98 MB | Beats 25.81%



Code | Python

1 class MyHashSet(object):

Code

Python Auto

1 class MyHashSet(object):

2

3 def __init__(self):

4 self.data = [False] * 1000001

5

6 def add(self, key):

7 self.data[key] = True

8

9 def remove(self, key):

10 self.data[key] = False

11

Saved

Ln 13, Col 30

Testcase

Test Result

You must run your code first

Search

09:53 02-09-2026